[Tech proposal] A Universal AIGC Protocol

Background & Goal

The rapid emergence of new AIGC capabilities has accelerated product iteration cycles dramatically. However, current business development cycles are lengthy, and the resulting AIGC assets exhibit low reusability, leading to overall inefficiency.

This project seeks to solve these issues by designing a universal AIGC protocol. The aim is to develop workflows that are modular and easily adaptable to multiple scenarios, ensuring that new use cases can be supported with minimal additional effort.

Before

- Each business defines its own workflow.
- Most workflow steps are highly customized and tightly coupled with specific business scenarios, limiting asset reuse.

After

- Workflows are deconstructed into a set of modular nodes.
- Each node functions as an isolated, reusable asset, enabling efficient reconstruction and adaptation for new scenarios.

Solution

Workflow definition:

A workflow can be presented by a **Directed Acyclic Graph (DAG)**

Each node of the graph represents one capability

Each edge of the graph defines the order of execution as well as the input & output constraints.

- **Client doesn't need to know the details of cloud workflow.** The interpreter on client should be able to interpret all the server steps into one single node.
- **Server doesn't need to know all client nodes either.** Server will use the request JSON sent by client to trim all client nodes and only execute the cloud workflow as a sub graph.

Each workflow is described by a directed acyclic graph,

- Each node presents an atomic client or server capability, and
- Each edge presents the input, output and execution orders.

Key Parts

1. Workflows are defined and hosted on Loki.
2. Loki will transform the complete workflow into sub-workflows, send them to Client and Server correspondingly.
3. Client and Server will each have their own task management framework responsible for executing the workflow and delegating tasks to corresponding node handlers.

A simplified workflow execution will look like below:

https://bytedance.us.larkoffice.com/sync/FXVUdKPfVsX2lEbTaZCusmdssog

E2E System Diagram

https://bytedance.us.larkoffice.com/sync/QjYrd8jJdshVy3bC0SfujVHDs9e

Workflow Entities Breakdown

Reusing the JSON format of ComfyUI

`workflow`

Key	Type	Enum Values	Note
nodes	JSON list of `node`		A complete representation of the workflow

`node`

Key	Type	Enum values	Note
unique ID	Int	1, 2, 3, ...	
class_type	string	faceMerge/HugAlgo/TnS/faceDetect	Defines functionality of the node
inputs			Defined by `class_type`.

			Raw input parameters or `connections` to other nodes
outputs			

`connections`

Nodes are linked using references

- "input_key": ["node_id", output_index]

e.g. `FaceMergeEffect` output is an image. An example node implementation would look like below:

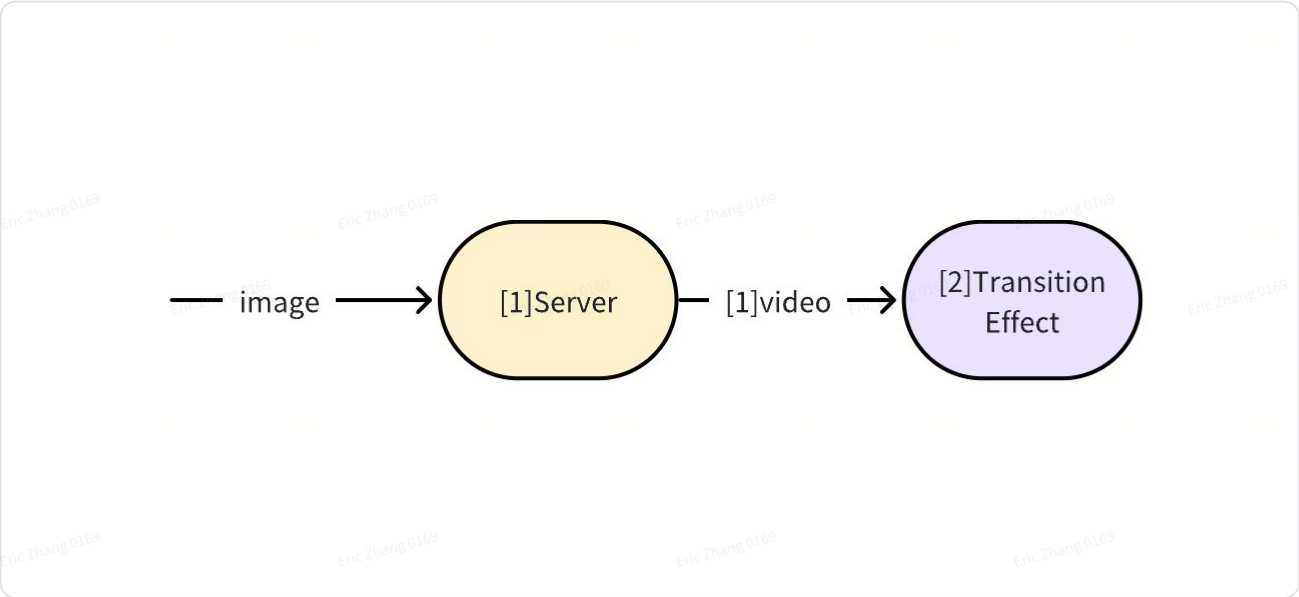
Code block

```
# import libs
class FaceMergeEffect(BaseModel):
    @classmethod
    def INPUT_TYPES(cls):
        return {
            "required": {

                "image_main": ("IMAGE",),  # Main face image
                "image_background": ("IMAGE",),  # Background image
                "source": ("STRING", {"default": "loki://79816635"}),
            }
        }

    RETURN_TYPES = ("IMAGE","gender")  # Output is an image
    FUNCTION = "merge_faces"
    CATEGORY = "Effects"

    def merge_faces(self, image_main, image_background, source, is_async):
        # function impl
        ##
        ## return (image, gender)

# Register the node with ComfyUI
register_node(FaceMergeEffect)
```

Example: Offline processing

> ✅Using standard comfyUI format

Client transformed workflow

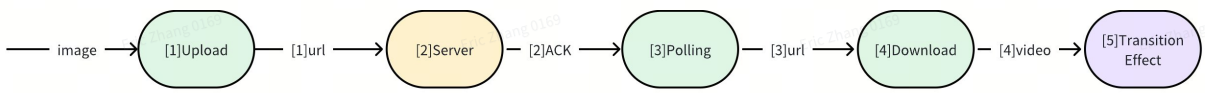


Code block

```
{
  "last_node_id": 1, // ID of the last node
  "last_link_id": 1, // ID of the last link
  "nodes": [
    {
      "id": 1, // server
      "type": "server",
      "inputs": [
        {
          "name": "image",
          "type": "url",
          "slot_index": 0 // Input slot index
        },
      ],
      "outputs": [
        {
          "name": "video",
          "type": "url",
          "links": [1],
          "slot_index": 0 // Output slot index
        }
    ]
    },
    {
      "id": 2, // transition effect
      "type": "clientTransitionEffect",
      "inputs": [
        {
```

```
            "name": "video",
            "type": "VIDEO",
            "link": 1,
            "slot_index": 0 // Input slot index
          },
        ],
        "outputs": [
          {
            "name": "postProcessedVideo",
            "type": "VIDEO",
            "links": [],
            "slot_index": 0 // Output slot index
          }
        ]
      },
    ],
    "links": [],
    "config": {}, // Global configuration settings (empty in this case)
    "extra": {
      "groupNodes": {} // Additional grouping information (empty in this case)
    }
  }

```

client transformed workflow + static nodes (upload, progress polling, download)



Code block

```
{
  "last_node_id": 5, // ID of the last node
```

```
  "last_link_id": 4, // ID of the last link
  "nodes": [
    {
      "id": 1, // upload
      "type": "uploadImage",
      "inputs": [
      {
          "name": "image",
          "type": "IMAGE",
          "slot_index": 0 // Input slot index
      }],
      "outputs": [
        {
          "name": "imageURL",
          "type": "URL",
          "links": [
            1
          ],
          "slot_index": 0 // Output slot index
        }
      ]
    },
    {
      "id": 2, // Server request
      "type": "server",
      "inputs": [
        {
          "name": "imageURL",
          "type": "URL",
          "link": 1,
          "slot_index": 0 // Input slot indexUpload
        },
      ],
      "outputs": [
      {
         "name": "ACK",
         "type": "Int",
         "link": [2],
         "slot_index": 0 // Output slot index
      }]
    },
    {
      "id": 3, // Progress polling
      "type": "progress",
      "inputs": [
        {
          "name": "ACK",
```

```
50            "type": "Int",
51            "link": 2,
52            "slot_index": 0 // Input slot index
53          },
54        ],
55        "outputs": [
56        {
57           "name": "ResultURL",
58           "type": "URL",
59           "link": [3],
60           "slot_index": 0 // Output slot index
61        }]
62      },
63      {
64        "id": 4, // Result download
65        "type": "download",
66        "inputs": [
67          {
68           "name": "ResultURL",
69           "type": "URL",
70           "link": 3,
71           "slot_index": 0 // Output slot index
72        }
73        ],
74        "outputs": [
75        {
76           "name": "Video",
77           "type": "VIDEO",
78           "link": [4],
79           "slot_index": 0 // Output slot index
80        }]
81      },
82      {
83        "id": 5, // transition effect
84        "type": "clientTransitionEffect",
85        "inputs": [
86          {
87            "name": "Video",
88            "type": "VIDEO",
89            "link": 4,
90            "slot_index": 0 // Input slot index
91          },
92        ],
93        "outputs": [
94          {
95            "name": "PostProcessedVideo",
96            "type": "VIDEO",
```

```
            "links": [],
            "slot_index": 0 // Output slot index
          }
        ]
      },
    ],
    "links": [
      [1, 1, 0, 2, 0, "URL"], // Connects Upload (Node 1, first output, type URL) to Server (Node 2, first input)
      [2, 2, 0, 3, 0, "Int"]  // Connects Server (Node 2, frist output, type Int) to Progress Polling (Node 3, first input)
      [3, 3, 0, 4, 0, "URL"]  // Connects Progress Polling (Node 3,  first output, type URL) to Download (Node 4, first input)
      [4, 4, 0, 5, 0, "Video"]  // Connects Download (Node 4,  first output, type Video) to clientTransitionEffect (Node 5, first input)
    ],
    "config": {}, // Global configuration settings (empty in this case)
    "extra": {
      "groupNodes": {} // Additional grouping information (empty in this case)
    }
  }

```

Example: AI self

> ⚠Not using standard comfyUI format, just for reference

Core Field	Description	structs	Required	Sample
business_scene	Define scene by business	string	Yes	effect_aigc
task_type	Define type by task	string	Yes	image2video
algo_raw	Algo raw input data	struct	No	req_key: request service
				req_json: algo config
image_infos	Input image infos	list	No	uri: 12345
text	User input test	list	No	text_id: 12345
video	User upload videos/record videos	list	No	vid: 12345
sound	User voices or music	list	No	clip_id: 12345

https://bytedance.us.larkoffice.com/sync/FXVUdKPfVsX2lEbTaZCusmdssog

Complete Workflow

Code block

```
{
  "nodes": {
  {
      "id": 1,
      "class_type": "ClientUserPrompt",
      "inputs": {
      }
      "outputs": {
         "prompt": "text",
      }
    },
    {
      "id": 2,
      "class_type": "ClientFaceDetectBachAPI",
      "inputs": {
        "image": "input_image_2",
      }
      "outputs": {
         "image": "outout_image_2",
      }
    },
    {
      "id": 3,
      "class_type": "ClientFilter",
      "source": "loki://323432423",
      "inputs": {
        "image": ["2", 0],
      }
      "outputs": {
         "image": "outout_image_3",
      }
    },
    {
      "id:": 4,
      "class_type": "ServerAlgo",
      "inputs": {
         "image": [["3", 0], ["2", 0]],
         "text": ["1", 0],
      },
      "outputs": {
         "image": "outout_image_4",
      }
```

```
      },
      {
        "id:": 5,
        "class_type": "ServerEffectCloudEditing",
        "inputs": {
           "image": [["4", 0]],
           "text": ["1", 0],
        },
        "outputs": {
           "image": "outout_image_5",
        }
      },
      {
        "id": 6
        "class_type": "ClientTransitionEffect",
        "source": "loki://45643524",
        "inputs": {
          "image": ["5", 0],
        }
        "outputs": {
           "video": "outout_video_6",
        }
      },
    }
  }
```

Notes

Some nodes are not explicitly shown in the workflow since they are always present in every workflow. These nodes are standardized, and we have offloaded their capabilities to the AIGC arch as core service providers.，e.g.

1. [Server]Content moderation flow: Content moderation flow won't show on the workflow and AIGC arch will provide generic moderation through each compliance process, e.g. CSAM, minorSafety, etc.

Code block

```
{
    "AImeUploadImage": {
        "ssh_Self_harm_challenges_nopolicy": 1.0,
        "ssh_violation": 0.90,
        "ssh_multimodal_v1_violation" : 0.999,
        "ssh_end2end_v2_violation" : 0.999,
        "iarg_v4_iarg_e2e_v4_weapon": 1.0,
        "ansa_hydra_porn": 0.94,
        "gacha_porn": 0.95,
```

```
            "extreme_risk_csam_v2_nsn_prob": 0.4623,
            "extreme_risk_csam_v2_ncmec_prob": 0.01355
        }
    }
```

2. [Client] media upload & download
3. [Client] network wrapper

Projects breakdown

Team members

Android: Hardik Shah (1) Kristoffer Solomon (1) Yichen Zhu (~0.5)

iOS: Eric Zhang (1) Chen He Zhechao Lyu (~0.5)

Key Direction	Priority	Sizing	POC	Tasks breakdown	More details	Notes
						1. Build from scratch, or refactor current task manager?

Reference

1. [WIP Proposal]: 多场景可复用的AIGC通用协议
2. AIGC Arch Client / Effect Foundation
3. [WIP] AIGC Cloud Workflow
4. AIGC 协议

体验文档

Effect i2v AIGC Offline Processing 体验反馈 Offline Processing One Pager

Live photo: AI Livephoto One Pager [AI LIVEPHOTO] 体验指南 https://tea-va.bytedance.net/tea/app/55/dashboard/7477464726344368695

T2I/I2V: Editor Pro T2I/I2V Testing Guide EP Text to Image/Video Server Tech Solution EditorPro Text to Image/Video 文生图/视频 Client- Tech Solution

Editor Pro- [Magic异步链路改造&增加i2v玩法] -iOS-Tech Solution

Discussions

Workflow manager

建议新建库，依赖难解，creativeArch同一层

1. 协议格式敲定
   a. comfyUI？
2. **节点类的实现方法**
   a. 哪里注册，存在哪里?
   b. 自动脚本
   c. ~~特效包改造，跳过这些直接实现~~
3. 服务端节点在客户端的具体处理方式
   a. 节点转化
   b. 转化发生在loki还是客户端？tradeoff分析
      i. Loki可以完成转化
   c. 合规问题，哪些节点一定要服务端/客户端做。
   d. 模型进化后工作流进一步的简化
4. 节点具体定义
   a. class_type, base_class_type
   b. 上传下载是否也作为节点
5. 云处理
   a. 合规
   b. 资源
   c. 现在的blocker，为啥不能全部端转云
6. 补充
   a. 非特效包场景补充
   b. 主拍特效转编辑，改造

Loki 可视化功能，有成本，json编辑器，可做定制下发，有校验能力

挂载信息，无ai标识，哪些视频是根据这个协议操作，打标

检验功能

Scenarios breakdown

Shooting Effect

[Draft]Tiktok AIGC Effect 组件化设计- 特效方案

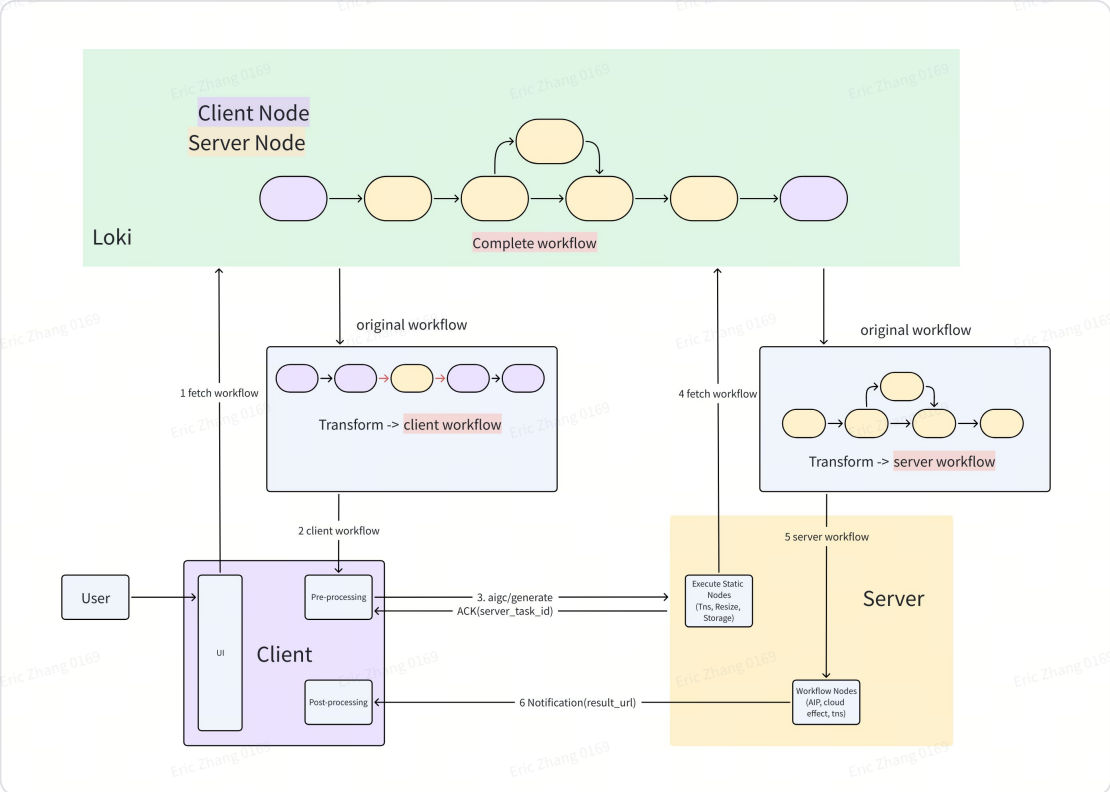
Each workflow is described by a directed acyclic graph,

- Each node presents an atomic client or server capability, and
- Each edge presents the input, output and execution orders.

Key Parts

1. Workflows are defined and hosted on Loki.
2. Loki will transform the complete workflow into sub-workflows, send them to Client and Server correspondingly.
3. Client and Server will each have their own task management framework responsible for executing the workflow and delegating tasks to corresponding node handlers.

A simplified workflow execution will look like below:



Dependencies & Timeline

Dependencies	Timeline	POC team

How to host the protocol		Loki, Server, TTEH
Server cloud-processing implementation (AI algo + cloudflow (editing, etc))		Server, Effect production
Client task management framework refactor		EP Client
Effect Platform, download JSON + effect package		Effect Platform, Client
Effect package refactor		IE + Client

1. Finalize protocol & entity definition.
2. How to store, filter, download the protocol
3. How each capability provider exports a node definition.
    a. Provide a script to auto generate the node, for each provider
    b. What protocol should it follow
4. IE refactor
    a. Not necessarily depends on #2 and #3

[技术提案]通用AIGC协议

背景与目标

新的AIGC能力的迅速涌现，极大地加速了产品迭代周期。然而，当前的业务开发周期冗长，由此产生的AIGC资产复用性低，导致整体效率低下。

本项目旨在通过设计通用的AIGC协议来解决这些问题。目标是开发模块化且易于适应多种场景的工作流程，确保新用例能够以最少的额外工作量得到支持。

之前

- 每个企业都定义自己的工作流程。
- 大多数工作流步骤都是高度定制的，并且与特定业务场景紧密耦合，限制了资产复用。

之后

- 工作流被解构为一组模块化节点。
- 每个节点都作为一个独立的、可复用的资产发挥作用，从而能够为新场景进行高效的重构和适配。

解决方案

工作流定义：

工作流可以用**有向无环图（DAG）来表示**

图的每个节点代表一种能力

图的每条边都定义了执行顺序以及输入和输出约束。

- **客户端无需了解云工作流的细节。**客户端上的解释器应能够将所有服务器步骤解释为单个节点。
- **服务器也不需要了解所有客户端节点。**服务器将使用客户端发送的请求JSON来修剪所有客户端节点，仅将云工作流作为子图执行。

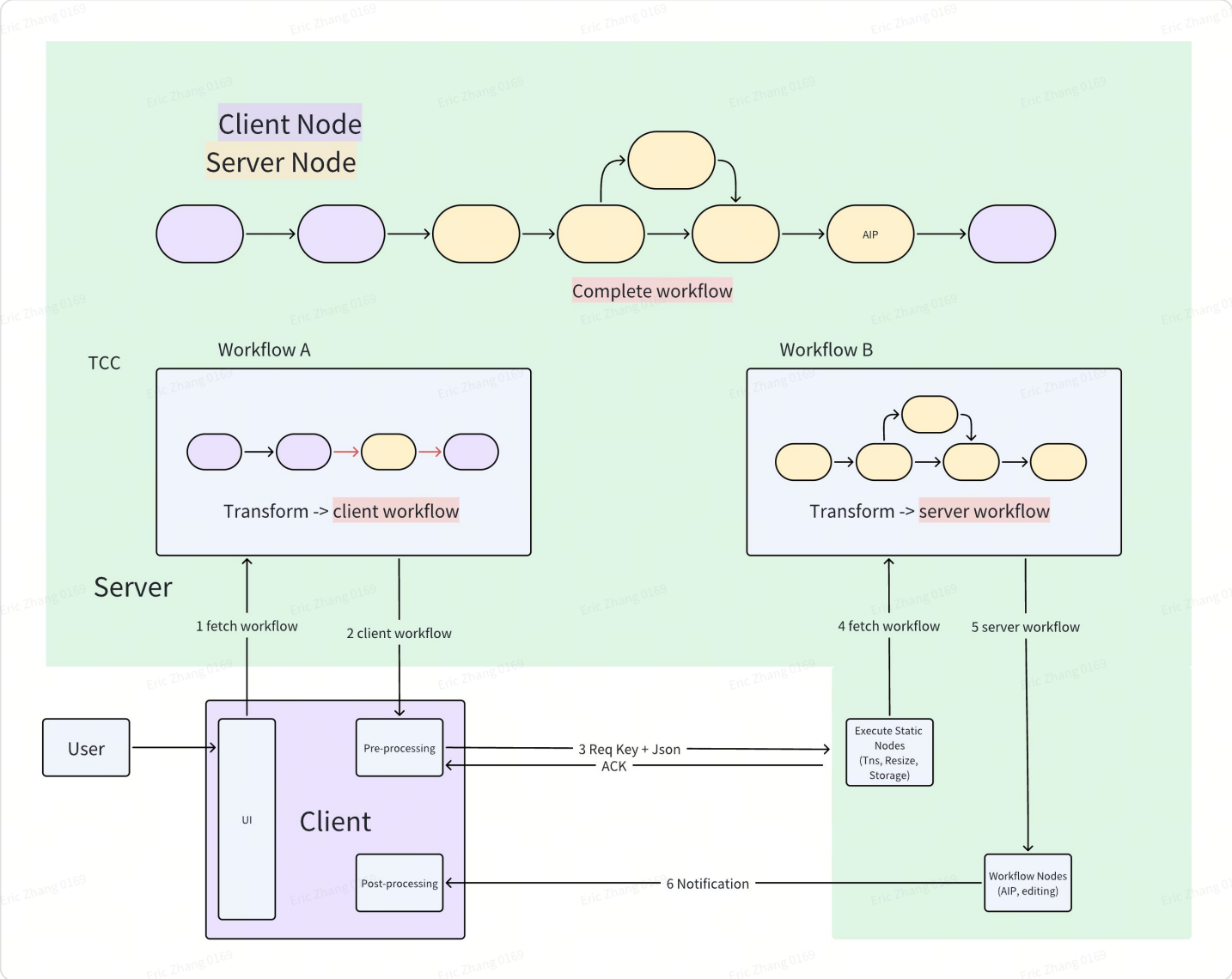
每个工作流都由有向无环图描述，

- 每个节点代表一个原子客户端或服务器功能，并且
- 每条边表示输入、输出和执行顺序。

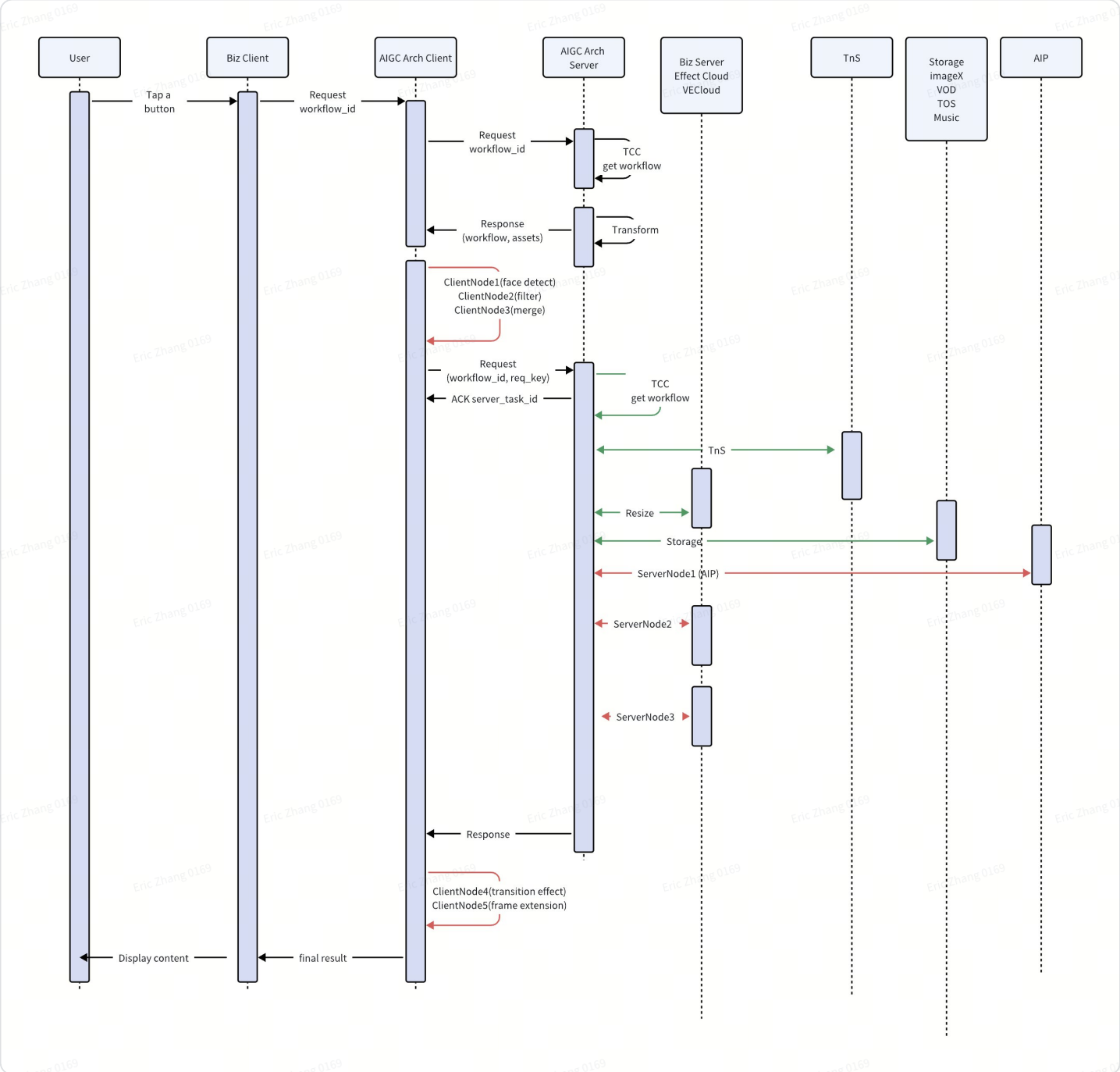
关键部件

1. 工作流在Loki上定义和托管。
2. 洛基将把完整的工作流转换为子工作流，并将它们分别发送到客户端和服务器。
3. 客户端和服务器将各自拥有自己的任务管理框架，负责执行工作流并将任务委派给相应的节点处理程序。

简化的工作流执行将如下所示：


Client Node
Server Node
AIP
Complete workflow
TCC
Workflow A
Transform -> client workflow
Workflow B
Transform -> server workflow
Server
1 fetch workflow
2 client workflow
4 fetch workflow
5 server workflow
User
UI
Client
Pre-processing
Post-processing
3 Req Key + Json
ACK
Execute Static Nodes (Tns, Resize, Storage)
6 Notification
Workflow Nodes (AIP, editing)

E2E系统图



工作流实体细分

复用ComfyUI的JSON格式

工作流

键	类型	枚举值	注意
节点	节点 `的JSON列表`		工作流的完整表示

节点

键	类型	枚举值	注意
唯一ID	整数	1, 2, 3, ...	
类类型	字符串	faceMerge/HugAlgo/TnS/faceDetect	定义节点的功能
输入			由 `class_type` 定义。 原始输入参数 `或` 与其他节点的连接
输出			

`连接`

节点通过引用进行链接

- "input_key": ["node_id", output_index]

例如 `FaceMergeEffect` 的输出是一张图像。一个示例节点实现如下所示：

Code block

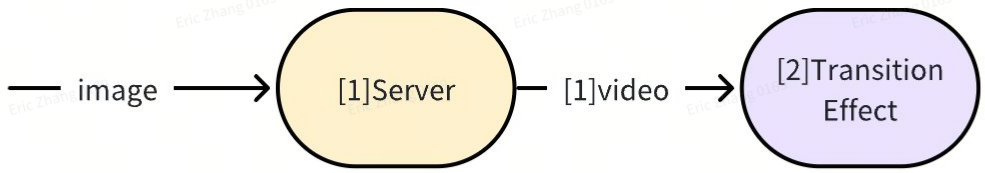
```
# import libs
class FaceMergeEffect(BaseModel):
    @classmethod
    def INPUT_TYPES(cls):
        return {
            "required": {

                "image_main": ("IMAGE",),  # Main face image
                "image_background": ("IMAGE",),  # Background image
                "source": ("STRING", {"default": "loki://79816635"}),
            }
        }

    RETURN_TYPES = ("IMAGE","gender")  # Output is an image
    FUNCTION = "merge_faces"
    CATEGORY = "Effects"

    def merge_faces(self, image_main, image_background, source, is_async):
        # function impl
        ##
        ## return (image, gender)

# Register the node with ComfyUI
```

```
register_node(FaceMergeEffect)
```

示例：离线处理

> ✅使用标准的ComfyUI格式

客户转变工作流程

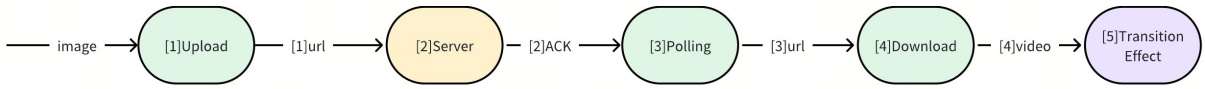


Code block

```
{
  "last_node_id": 1, // ID of the last node
  "last_link_id": 1, // ID of the last link
  "nodes": [
    {
      "id": 1, // server
      "type": "server",
      "inputs": [
        {
          "name": "image",
          "type": "url",
          "slot_index": 0 // Input slot index
        },
      ],
      "outputs": [
        {
          "name": "video",
          "type": "url",
          "links": [1],
          "slot_index": 0 // Output slot index
```

```
21          }
22        ]
23      },
24      {
25        "id": 2, // transition effect
26        "type": "clientTransitionEffect",
27        "inputs": [
28          {
29            "name": "video",
30            "type": "VIDEO",
31            "link": 1,
32            "slot_index": 0 // Input slot index
33          },
34        ],
35        "outputs": [
36          {
37            "name": "postProcessedVideo",
38            "type": "VIDEO",
39            "links": [],
40            "slot_index": 0 // Output slot index
41          }
42        ]
43      },
44    ],
45    "links": [],
46    "config": {}, // Global configuration settings (empty in this case)
47    "extra": {
48      "groupNodes": {} // Additional grouping information (empty in this case)
49    }
50  }
51
```

客户端转换工作流 + 静态节点（上传、进度轮询、下载）



Code block

```
{
  "last_node_id": 5, // ID of the last node
  "last_link_id": 4, // ID of the last link
  "nodes": [
    {
      "id": 1, // upload
      "type": "uploadImage",
      "inputs": [
      {
          "name": "image",
          "type": "IMAGE",
          "slot_index": 0 // Input slot index
      }],
      "outputs": [
        {
          "name": "imageURL",
          "type": "URL",
          "links": [
            1
          ],
          "slot_index": 0 // Output slot index
        }
      ]
    },
    {
      "id": 2, // Server request
      "type": "server",
      "inputs": [
        {
```

```
30            "name": "imageURL",
31            "type": "URL",
32            "link": 1,
33            "slot_index": 0 // Input slot indexUpload
34          },
35        ],
36        "outputs": [
37        {
38           "name": "ACK",
39           "type": "Int",
40           "link": [2],
41           "slot_index": 0 // Output slot index
42        }]
43      },
44      {
45        "id": 3, // Progress polling
46        "type": "progress",
47        "inputs": [
48          {
49            "name": "ACK",
50            "type": "Int",
51            "link": 2,
52            "slot_index": 0 // Input slot index
53          },
54        ],
55        "outputs": [
56        {
57           "name": "ResultURL",
58           "type": "URL",
59           "link": [3],
60           "slot_index": 0 // Output slot index
61        }]
62      },
63      {
64        "id": 4, // Result download
65        "type": "download",
66        "inputs": [
67          {
68           "name": "ResultURL",
69           "type": "URL",
70           "link": 3,
71           "slot_index": 0 // Output slot index
72        }
73        ],
74        "outputs": [
75        {
76           "name": "Video",
```

```
          "type": "VIDEO",
          "link": [4],
          "slot_index": 0 // Output slot index
        }]
      },
      {
        "id": 5, // transition effect
        "type": "clientTransitionEffect",
        "inputs": [
          {
            "name": "Video",
            "type": "VIDEO",
            "link": 4,
            "slot_index": 0 // Input slot index
          },
        ],
        "outputs": [
          {
            "name": "PostProcessedVideo",
            "type": "VIDEO",
            "links": [],
            "slot_index": 0 // Output slot index
          }
        ]
      },
    ],
    "links": [
      [1, 1, 0, 2, 0, "URL"], // Connects Upload (Node 1, first output, type URL) to Server (Node 2, first input)
      [2, 2, 0, 3, 0, "Int"]  // Connects Server (Node 2, frist output, type Int) to Progress Polling (Node 3, first input)
      [3, 3, 0, 4, 0, "URL"]  // Connects Progress Polling (Node 3,  first output, type URL) to Download (Node 4, first input)
      [4, 4, 0, 5, 0, "Video"]  // Connects Download (Node 4,  first output, type Video) to clientTransitionEffect (Node 5, first input)
    ],
    "config": {}, // Global configuration settings (empty in this case)
    "extra": {
      "groupNodes": {} // Additional grouping information (empty in this case)
    }
  }

```

示例：AI自我

> ⚠未使用标准ComfyUI格式，仅供参考

核心领域	描述	结构体	必填	示例
业务场景	按业务定义场景	字符串	是的	效果_AIGC
任务类型	按任务定义类型	字符串	是的	图像转视频
算法原始	算法原始输入数据	结构体	不	req_key: 请求服务
				req_json: 算法配置
图像信息	输入图像信息	列表	不	统一资源标识符: 12345
文本	用户输入测试	列表	不	文本ID: 12345
视频	用户上传视频/录制视频	列表	不	视频编号：12345
声音	用户声音或音乐	列表	不	剪辑ID: 12345

完整工作流程

Code block

```
{
  "nodes": {
  {
      "id": 1,
      "class_type": "ClientUserPrompt",
      "inputs": {
      }
      "outputs": {
         "prompt": "text",
      }
    },
    {
      "id": 2,
      "class_type": "ClientFaceDetectBachAPI",
      "inputs": {
        "image": "input_image_2",
      }
      "outputs": {
         "image": "outout_image_2",
      }
    },
    {
      "id": 3,
```

```
24          "class_type": "ClientFilter",
25          "source": "loki://323432423",
26          "inputs": {
27            "image": ["2", 0],
28          }
29          "outputs": {
30             "image": "outout_image_3",
31          }
32        },
33        {
34          "id:": 4,
35          "class_type": "ServerAlgo",
36          "inputs": {
37             "image": [["3", 0], ["2", 0]],
38             "text": ["1", 0],
39          },
40          "outputs": {
41             "image": "outout_image_4",
42          }
43        },
44        {
45          "id:": 5,
46          "class_type": "ServerEffectCloudEditing",
47          "inputs": {
48             "image": [["4", 0]],
49             "text": ["1", 0],
50          },
51          "outputs": {
52             "image": "outout_image_5",
53          }
54        },
55        {
56          "id": 6
57          "class_type": "ClientTransitionEffect",
58          "source": "loki://45643524",
59          "inputs": {
60            "image": ["5", 0],
61          }
62          "outputs": {
63             "video": "outout_video_6",
64          }
65        },
66      }
67    }
```

注释

某些节点在工作流中未明确显示，因为它们在每个工作流中始终存在。这些节点是标准化的，我们已将其功能作为核心服务提供者卸载到AIGC架构中，例如

1. [服务器]内容审核流程：内容审核流程不会显示在工作流中，AIGC架构将通过每个合规流程（如CSAM、未成年人安全等）提供通用审核。

Code block

```
{
    "AImeUploadImage": {
        "ssh_Self_harm_challenges_nopolicy": 1.0,
        "ssh_violation": 0.90,
        "ssh_multimodal_v1_violation" : 0.999,
        "ssh_end2end_v2_violation" : 0.999,
        "iarg_v4_iarg_e2e_v4_weapon": 1.0,
        "ansa_hydra_porn": 0.94,
        "gacha_porn": 0.95,
        "extreme_risk_csam_v2_nsn_prob": 0.4623,
        "extreme_risk_csam_v2_ncmec_prob": 0.01355
    }
}
```

2. [客户端]媒体上传和下载
3. [客户端]网络包装器

项目细分

团队成员

安卓： Hardik Shah (1) Kristoffer Solomon (1) Yichen Zhu (~0.5)

iOS： Eric Zhang (1) Chen He Zhechao Lyu (~0.5)

关键方向	优先级	尺寸	对接人	任务分解	更多详情	注释

						1. 从头开始构建，还是重构当前的任务管理器?

参考

1. [WIP Proposal]: 多场景可复用的AIGC通用协议
2. AIGC架构客户端 / 效果基金会
3. [WIP] AIGC云工作流
4. AIGC协议

体验文档

效果i2v AIGC Offline Processing 体验反馈 Offline Processing One Pager

动态照片： AI Alive One Pager [AI Alive] 体验指南 https://tea-va.bytedance.net/tea/app/55/dashboard/7477464726344368695

T2I/I2V： Editor Pro T2I/I2V Testing Guide EP Text to Image/Video Server Tech Solution EditorPro Text to Image/Video 文生图/视频 Client- Tech Solution

Editor Pro- [Magic异步链路改造&增加i2v玩法] -iOS-Tech Solution

讨论

工作流管理器

建议新建库，依赖难解，creativeArch同一层

1. 协议格式敲定
    a. ComfyUI?
2. **节点类的实现方法**
    a. 哪里注册，存在哪里?
    b. 自动脚本
    c. ~~特效包改造，跳过这些直接实现~~
3. 服务端节点在客户端的具体处理方式
    a. 节点转化
    b. 转化发生在loki还是客户端？权衡分析
        i. Loki可以完成转化
    c. 合规问题，哪些节点一定要服务端/客户端做。
    d. 模型进化后工作流进一步的简化
4. 节点具体定义
    a. 类类型，基类类型
    b. 上传下载是否也作为节点
5. 云处理
    a. 合规
    b. 资源
    c. 现在的blocker，为啥不能全部端转云

6. 补充
    a. 非特效包场景补充
    b. 主拍特效转编辑，改造

Loki 可视化功能，有成本，json编辑器，可做定制下发，有校验能力

挂载信息，无ai标识，哪些视频是根据这个协议操作，打标

检验功能

场景细分

射击特效

[草稿]TikTok AIGC特效组件化设计 - 特效方案

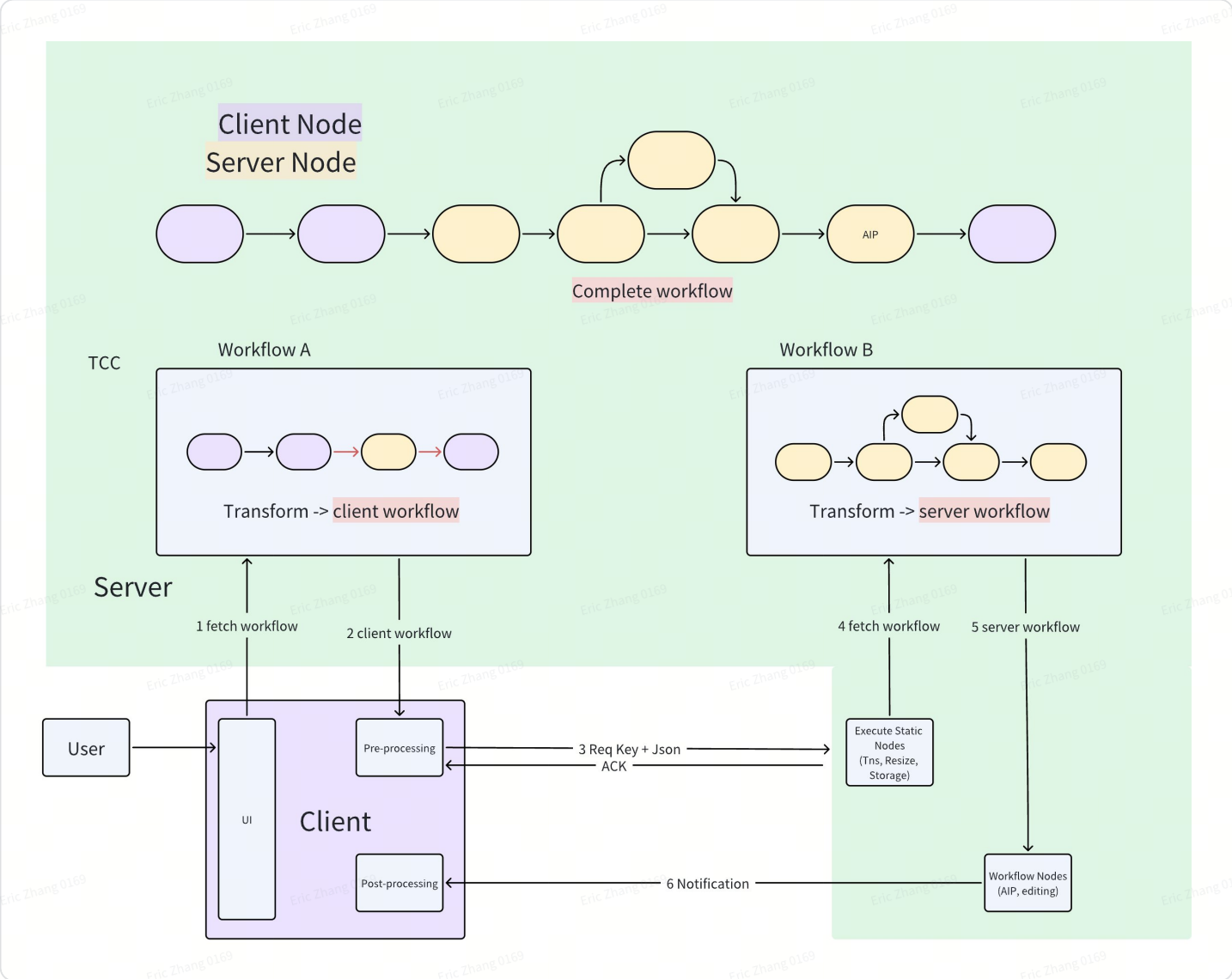
每个工作流都由有向无环图描述，

- 每个节点代表一个原子客户端或服务器功能，并且
- 每条边表示输入、输出和执行顺序。

关键部件

1. 工作流在Loki上定义和托管。
2. 洛基将把完整的工作流转换为子工作流，并将它们分别发送到客户端和服务器。
3. 客户端和服务器将各自拥有自己的任务管理框架，负责执行工作流并将任务委派给相应的节点处理程序。

简化的工作流执行将如下所示：



依赖项与时间线

依赖项	时间线	POC团队
如何托管协议		洛基、服务器、TTEH
服务器云处理实现 (AI算法 + 云流程（编辑等）)		服务器，特效制作
客户端任务管理框架重构		EP客户端
特效平台，下载JSON+资源包		特效平台，客户端
资源包重构		IE + 客户端

1. 完成协议和实体定义。
2. 如何存储、筛选、下载协议
3. 每个能力提供者如何导出节点定义。

   a. 为每个供应商提供一个自动生成节点的脚本
   b. 它应该遵循什么协议
4. IE重构
   a. 不一定取决于 #2 和 #3